

CRM LiVrai — état des lieux avant refonte

Analyse de l'application de gestion des livraisons en production :
périmètre fonctionnel, architecture, et capacité à absorber la
croissance de l'activité.

Commanditaire Meilin, chef de projet Périmètre Application CRM existante
Version analysée `app 0.0.1-SNAPSHOT` Date Septembre 2026

01

Contexte et périmètre

LiVrai exploite un CRM interne pour gérer ses commandes de livraison B2B. La croissance de l'entreprise met cette application sous tension. Cet audit établit ce qu'elle fait, comment elle est construite, et où elle cédera si l'activité continue d'augmenter.

La situation

LiVrai est une entreprise de livraison B2B en forte croissance : le nombre de clients et le volume de commandes augmentent rapidement. L'application CRM qui soutient cette activité a été conçue pour une échelle plus modeste, et les équipes en ressentent aujourd'hui les limites.

Trois signaux ont motivé cet audit. Ils ne sont pas de même nature :

SIGNAL	NATURE	CE QUE L'AUDIT DOIT PRODUIRE
Lenteurs	CONSTATÉ	Identifier le mécanisme technique qui les produit, et à quel volume elles s'aggravent.
Maintenance difficile	CONSTATÉ	Nommer les caractéristiques du code qui rendent chaque évolution coûteuse et risquée.

Disponibilité en pic	ANTICIPÉ	Aucun incident n'a encore eu lieu. Il faut donc démontrer par un scénario ce qui se produirait, et à quel seuil.
----------------------	----------	---

Cette distinction gouverne la méthode du rapport. Pour les deux premiers points, on remonte du symptôme vers sa cause dans le code. Pour le troisième, aucun fait n'est observable : on descend de l'architecture vers le scénario de rupture.

Objectif de l'audit

Ce document est un **état des lieux**, pas une proposition de refonte. Son rôle est de fournir à la direction de LiVrai une base factuelle pour décider d'un investissement technique — de la même façon qu'un diagnostic précède des travaux sans décider à la place du propriétaire. Les solutions ne sont pas prescrites ici ; les problèmes sont qualifiés de manière à ce que les options de refonte puissent être arbitrées ensuite.

Périmètre

INCLUS	EXCLU
Code source applicatif (app/)	Infrastructure d'hébergement et supervision (non communiquées)
Schéma de base de données (db/)	Métriques de production : temps de réponse réels, nombre d'utilisateurs, volumétrie actuelle
Dépendances et configuration de build	Entretiens utilisateurs
Parcours utilisateur exécutés sur l'application en fonctionnement	Conception de la cible

Méthode

L'audit combine deux approches. L'analyse statique du code source, du schéma de base et des descripteurs de déploiement établit l'architecture et ses propriétés structurelles. **L'application a par ailleurs été déployée et parcourue** — MySQL 8 en conteneur, Tomcat 8.5, archive WAR déployée sur localhost:8080/app — afin d'exercer les deux profils utilisateurs sur l'ensemble des parcours.

Ce second volet a permis de vérifier les comportements par l'observation plutôt que par la lecture. Trois défauts identifiés lors de l'analyse du code ont ainsi été **confirmés en fonctionnement**, et plusieurs constats supplémentaires en sont issus. Les observations correspondantes sont signalées dans le rapport.

LIMITE MÉTHODOLOGIQUE

Les parcours ont été exécutés sur un environnement de test alimenté par des données créées via l'interface, sans accès à la production ni aux utilisateurs réels. Les constats fonctionnels et structurels sont donc établis avec certitude, mais **les seuils chiffrés de rupture restent des ordres de grandeur** : les confirmer suppose un test de charge sur une copie représentative de la base, recommandé en préalable à la refonte.

02

Vue fonctionnelle

Le CRM couvre le cycle de vie d'une commande de livraison, de sa création par le client jusqu'à sa facturation par l'administrateur. Deux profils d'utilisateurs, sept écrans, aucune fonction annexe.

Les deux profils utilisateurs

Le modèle de droits repose sur un unique booléen `admin` dans la table `user`. Il n'existe donc que deux rôles, sans granularité intermédiaire :

PROFIL	QUI	CE QU'IL PEUT FAIRE
Client <code>ADMIN = FALSE</code>	Entreprise cliente de LiVrai	Se connecter · Commander une livraison (volume, poids) · Consulter ses propres livraisons, à venir et passées
Administrateur <code>ADMIN = TRUE</code>	Équipe interne LiVrai	Tout ce qui précède · Voir les livraisons de tous les clients · Accepter / refuser une commande · Facturer une livraison acceptée · Consulter la liste des clients · Créer un compte client

Cas d'utilisation

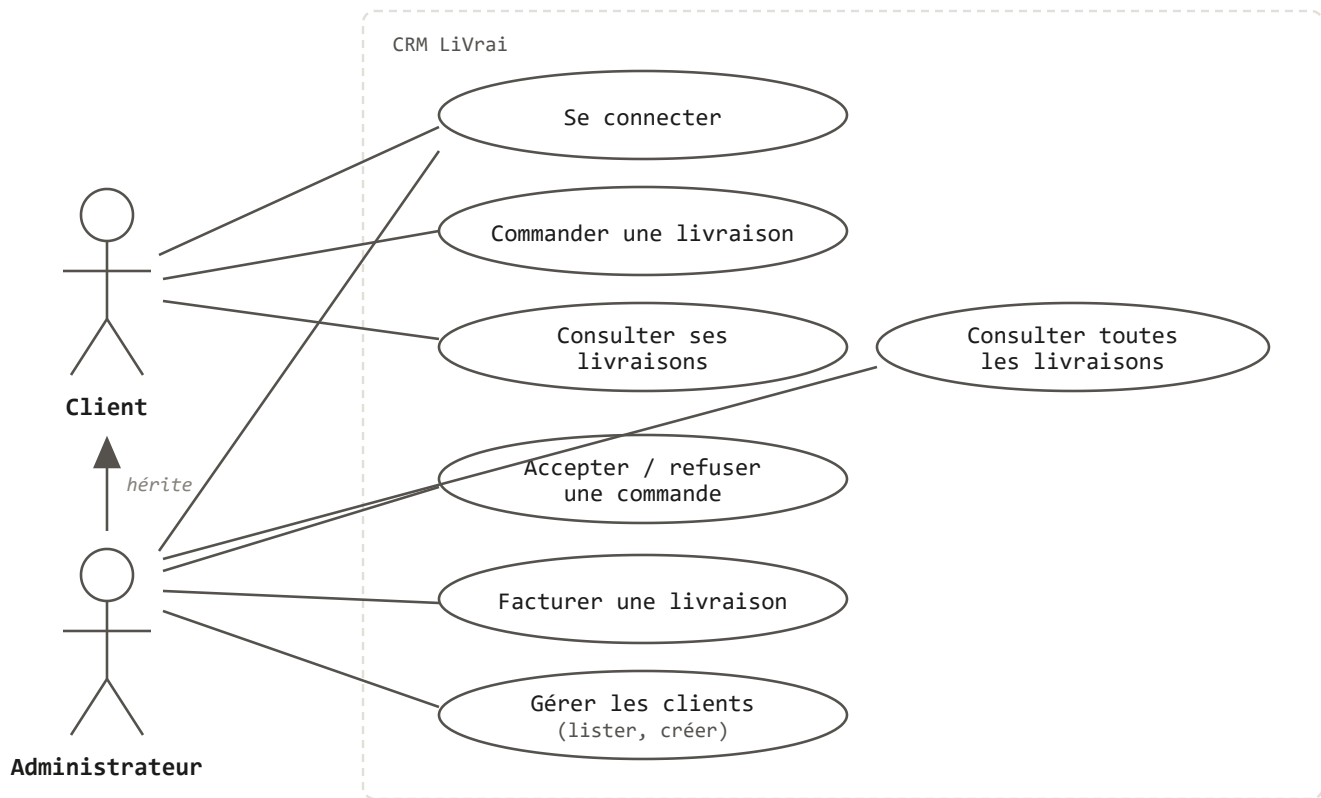


Diagramme de cas d'utilisation UML. L'administrateur hérite des capacités du client : le même compte peut commander et administrer. Sept cas d'utilisation au total.

Cycle de vie d'une livraison

Le statut d'une livraison est stocké en base sous forme de **chaîne de caractères en français** (`VARCHAR(255)`) : **"En attente"**, **"Acceptée"**, **"Refusée"**, **"Terminée"**. Quatre états, trois transitions, toutes déclenchées par l'administrateur.

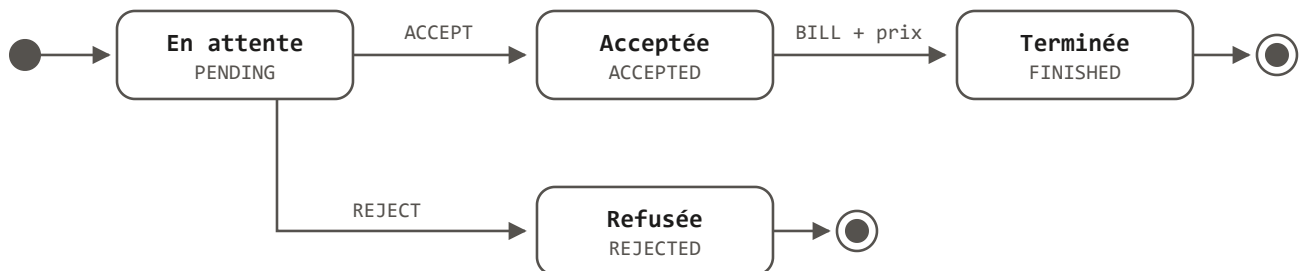


Diagramme d'états UML. Le prix n'est renseigné qu'à la facturation : une livraison acceptée mais non facturée porte un prix nul.

Parcours métier de bout en bout

Le diagramme suivant décrit le cheminement complet d'une commande, du besoin exprimé par le client jusqu'à la facturation. Il fait apparaître les deux acteurs et leurs responsabilités respectives, ainsi que le point de rupture constaté lors des tests.

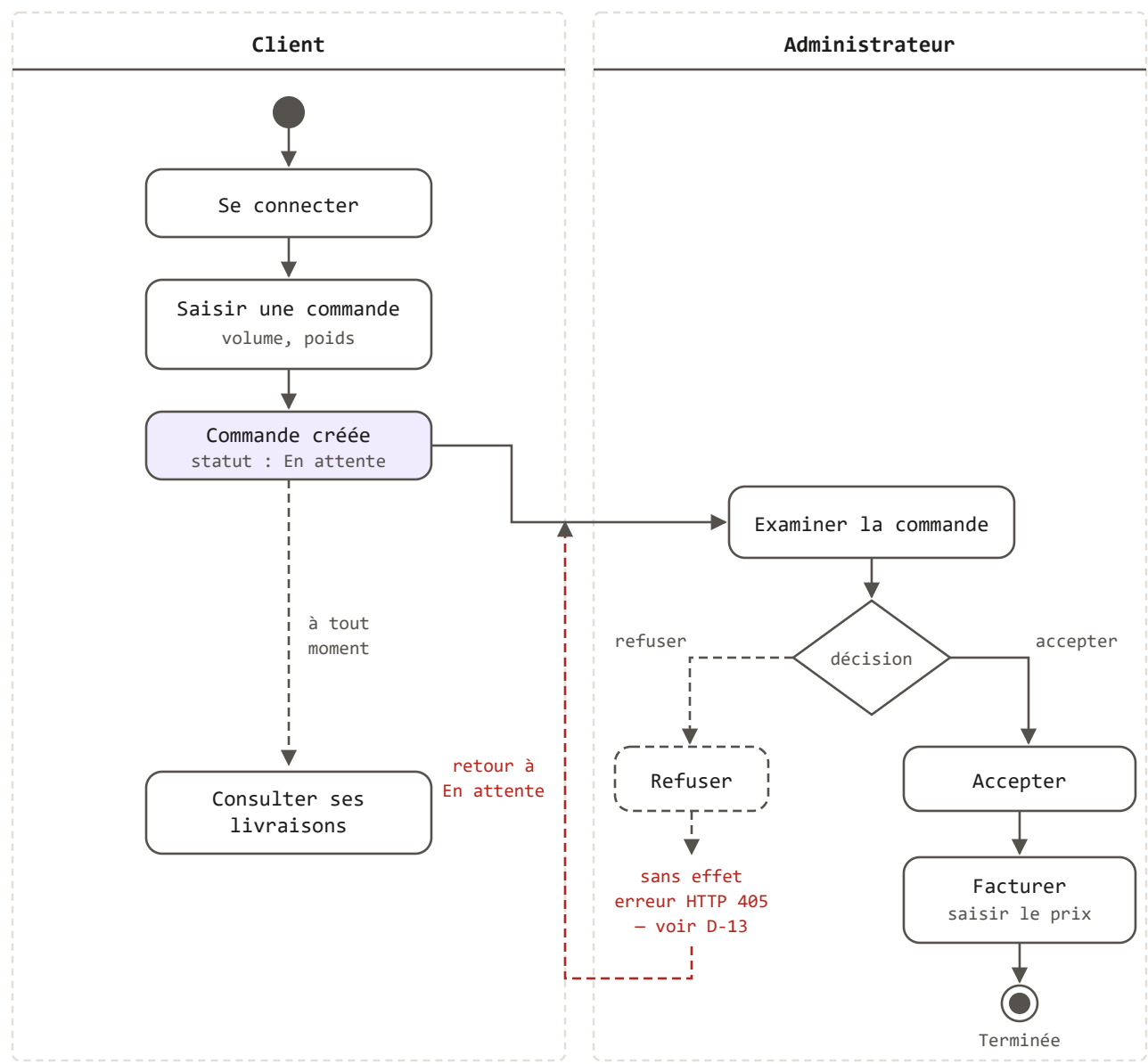
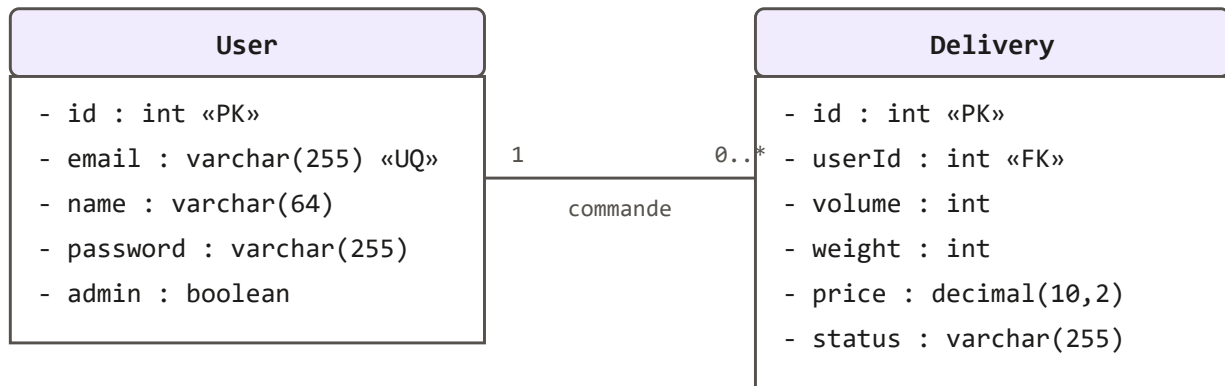


Diagramme d'activité UML avec couloirs. Le chemin « Refuser », représenté en pointillés, est présent dans l'interface mais **n'aboutit pas** : la commande reste « En attente ». Anomalie constatée lors des tests sur l'application en fonctionnement.

Modèle de données



Deux tables, une relation.

Aucun index sur `delivery.status`, critère de tri principal – voir D-04.

Diagramme de classes UML du modèle persisté. Le modèle est correctement normalisé et la contrainte d'intégrité référentielle est en place.

Ce que le CRM ne fait pas

L'absence de ces fonctions est un constat de périmètre, utile pour cadrer la refonte :

- Aucune recherche, aucun filtre, aucun tri sur les listes
- Aucune pagination : toutes les livraisons sont affichées d'un bloc
- Aucune modification ni suppression de livraison depuis l'interface
- Aucune gestion de l'inscription : les comptes clients sont créés par un administrateur
- Aucune réinitialisation de mot de passe
- Aucun export, aucun rapport, aucun tableau de bord
- Aucune notification, aucun envoi de courriel
- Aucune traçabilité des actions (qui a facturé quoi, et quand)

03

Expérience utilisateur

L'interface fonctionne, mais elle repose sur un modèle de navigation où chaque action recharge la page entière. Ce choix technique se traduit directement en temps d'attente pour les équipes —

et ce temps croît avec le volume de données.

Le parcours actuel

L'application compte sept pages JSP rendues côté serveur. Chaque interaction — clic sur un lien, envoi d'un formulaire, acceptation d'une commande — provoque un aller-retour complet avec le serveur, suivi du rechargement intégral de la page.

ÉCRAN	ROUTE	CONTENU ET INTERACTION
Connexion	/login	Formulaire email + mot de passe. En cas d'échec, la page est réaffichée sans message d'erreur .
Accueil	/	Page d'entrée après connexion.
Livraisons	/livraisons	Écran principal. Deux tableaux : à venir, passées. Pour l'admin, boutons Accepter / Refuser / Facturer sur chaque ligne.
Commande	/commande	Formulaire volume + poids.
Clients	/clients	Liste des clients et formulaire de création. Admin uniquement.
Déconnexion	/logout	Invalide la session.

Mes livraisons à venir

Id	Volume	Poids	Statut		
2	Ets Dupont	5	80	En attente	Accepter Refuser
3	Ets Dupont	40	950	En attente	Accepter Refuser
4	Ets Dupont	-99	-50	En attente	Accepter Refuser

Mes livraisons passées

Id	Volume	Poids	Prix	Statut
1	Ets Dupont	12	300 450.5	Terminée

Ecran `/livraisons` vu par un administrateur, sur l'application en fonctionnement. Trois anomalies y sont visibles simultanément : le décalage des colonnes (le nom du client apparaît sous l'en-tête « Volume »), une livraison de volume `-99` et de poids `-50` acceptée en base, et la fusion du poids et du prix dans une même cellule du tableau des livraisons passées (« 300 450.5 »).

Points de friction identifiés

UX-01 Rechargement complet à chaque action IMPACT FORT

Accepter une livraison recharge toute la liste. Sur un écran contenant plusieurs centaines de lignes, l'administrateur attend le rendu complet de la page pour valider une seule ligne — puis perd sa position dans le tableau et doit y revenir manuellement.

COÛT MÉTIER

Le traitement des commandes en lot devient une succession d'attentes. C'est la traduction directe des « lenteurs » signalées par les équipes.

UX-02 **Aucun retour visuel pendant le chargement** IMPACT MOYEN

Aucun indicateur de progression n'est affiché. Entre le clic et l'affichage, l'utilisateur ne dispose d'aucun signal : il ne sait pas si son action a été prise en compte.

CONSÉQUENCE OBSERVABLE

Double-clics sur les boutons d'action, donc risque de traitements en double — non protégé par le code.

UX-03 **Échec de connexion silencieux** IMPACT MOYEN

Dans `LoginServlet`, un identifiant erroné réaffiche simplement le formulaire. Aucun message n'explique l'échec. L'utilisateur ne sait pas s'il s'est trompé de mot de passe, si son compte est désactivé, ou si l'application est en panne.

UX-04 **Listes non paginées et non filtrables** IMPACT CRITIQUE À VOLUME

`getAllDeliveries()` retourne **toutes** les livraisons de la base, sans limite. L'écran devient inutilisable à mesure que l'historique s'accumule : l'admin doit faire défiler des milliers de lignes pour trouver une commande, sans recherche ni tri.

AGGRAVATION

Contrairement aux autres frictions, celle-ci s'intensifie mécaniquement avec la croissance de LiVrai. C'est le point de rencontre entre l'UX et le problème de volumétrie.

UX-05 **Défauts d'affichage dans la liste des livraisons** IMPACT FAIBLE

Deux anomalies concrètes relevées dans `deliveries.jsp` :

- La colonne « Client » de l'en-tête est conditionnée par `${!empty delivery.clientName}`, une variable qui n'existe pas à cet endroit du gabarit. L'en-tête ne s'affiche donc jamais, alors que la colonne de données, elle, apparaît — **les colonnes sont décalées pour l'administrateur**.
- Dans le tableau des livraisons passées, la balise `<td>` du prix est manquante : le prix est concaténé dans la cellule du poids.

CAUSE STRUCTURELLE

Les balises `<td>` ne sont jamais fermées dans le gabarit. Le navigateur corrige silencieusement, ce qui masque ce type d'erreur au développeur.

UX-06 Bouton « Refuser » adressé à la mauvaise route

DÉFAUT FONCTIONNEL

Toujours dans `deliveries.jsp` : le formulaire « Accepter » poste vers `/livraison` (le servlet d'action), tandis que le formulaire « Refuser » poste vers `/livraisons` — la page de liste, qui ne traite que les requêtes `GET`.

VÉRIFIÉ SUR L'APPLICATION

Test effectué sur l'instance en fonctionnement : l'action `ACCEPT` sur `/livraison` retourne `302` et la livraison passe au statut « Acceptée ». La même action `REJECT` sur `/livraisons` retourne `405 Method Not Allowed` et la livraison demeure « En attente » en base.

CONSÉQUENCE

Le refus d'une commande est impossible dans l'application. L'écran se recharge sans message d'erreur : l'administrateur peut croire l'action effectuée alors qu'elle n'a produit aucun effet.

NATURE

Ce n'est pas une limite d'architecture mais un **défaut fonctionnel en production**, à corriger indépendamment de la refonte.

UX-07 Intitulés d'écran inadaptés au profil administrateur

IMPACT FAIBLE

La barre de navigation s'adapte correctement au profil : le client voit « Commande » et « Livraisons », l'administrateur voit « Clients » et « Livraisons ». En revanche, les titres de la page des livraisons sont figés à « **Mes livraisons à venir** » et « **Mes livraisons passées** » pour les deux profils.

CONSTATÉ LORS DES TESTS

L'administrateur, qui consulte les livraisons de **tous** les clients, lit un intitulé qui suggère qu'il s'agit des siennes. L'ambiguïté est renforcée par le décalage de colonnes décrit en UX-05.

Accessibilité et ergonomie générale

Le style est intégralement embarqué dans chaque page via `<%@include file="style.css"%>`, ce qui empêche toute mise en cache navigateur de la présentation. Aucun élément d'accessibilité n'est présent : pas de `<label>` associé aux champs, pas d'attributs ARIA, pas d'ordre de tabulation défini. Aucune adaptation aux écrans mobiles n'est prévue.

04

Description technique

Application web Java monolithique construite sur l'API Servlet, avec rendu de pages JSP côté serveur et accès aux données en JDBC direct. Aucun framework applicatif. Déploiement en archive WAR sur un serveur Tomcat unique.

Socle technique et versions

COMPOSANT	VERSION EN PLACE	STATUT	OBSERVATION
Java (cible de compilation)	1.6	OBSOLÈTE	Défini dans <code>pom.xml</code> . Fin de support public en 2013. Interdit l'usage des lambdas, de l'API Stream, du <code>try-with-resources</code> et de tout apport ultérieur du langage.
Servlet API	<code>javax.servlet</code> 3.0.1	OBSOLÈTE	Spécification de 2009. L'écosystème Java a migré vers l'espace de noms <code>jakarta.*</code> : cette base est incompatible avec les serveurs d'applications récents.
Serveur d'application	Tomcat 8.5	FIN DE VIE	Support terminé en mars 2024. Ne reçoit plus de correctifs de sécurité.
JSP / JSTL	<code>jstl</code> 1.2, <code>taglibs-standard</code> 1.1.2	ANCIEN	Technologie de présentation serveur, sans évolution depuis plus d'une décennie.
Base de données	MySQL 8.x	À JOUR	Le seul composant du socle réellement supporté.
Pilote JDBC	<code>mysql-connector-java</code> 8.0.32	RÉCENT	Utilisé en JDBC brut, sans ORM ni pool de connexions.
Build	Maven, packaging <code>war</code>	STANDARD	Structure conventionnelle, six dépendances seulement.
Tests	JUnit 4.11 déclaré	INUTILISÉ	La dépendance figure au <code>pom.xml</code> mais aucun test n'existe dans le projet.

AUCUN FRAMEWORK APPLICATIF

L'application n'utilise ni Spring, ni Jakarta EE, ni Hibernate, ni aucun conteneur d'injection de dépendances. Tout — routage, sécurité, accès aux données, transactions — est écrit à la main sur l'API Servlet. Cela explique la compacité du code, mais aussi l'absence des garanties qu'un framework apporte par défaut.

Architecture en composants

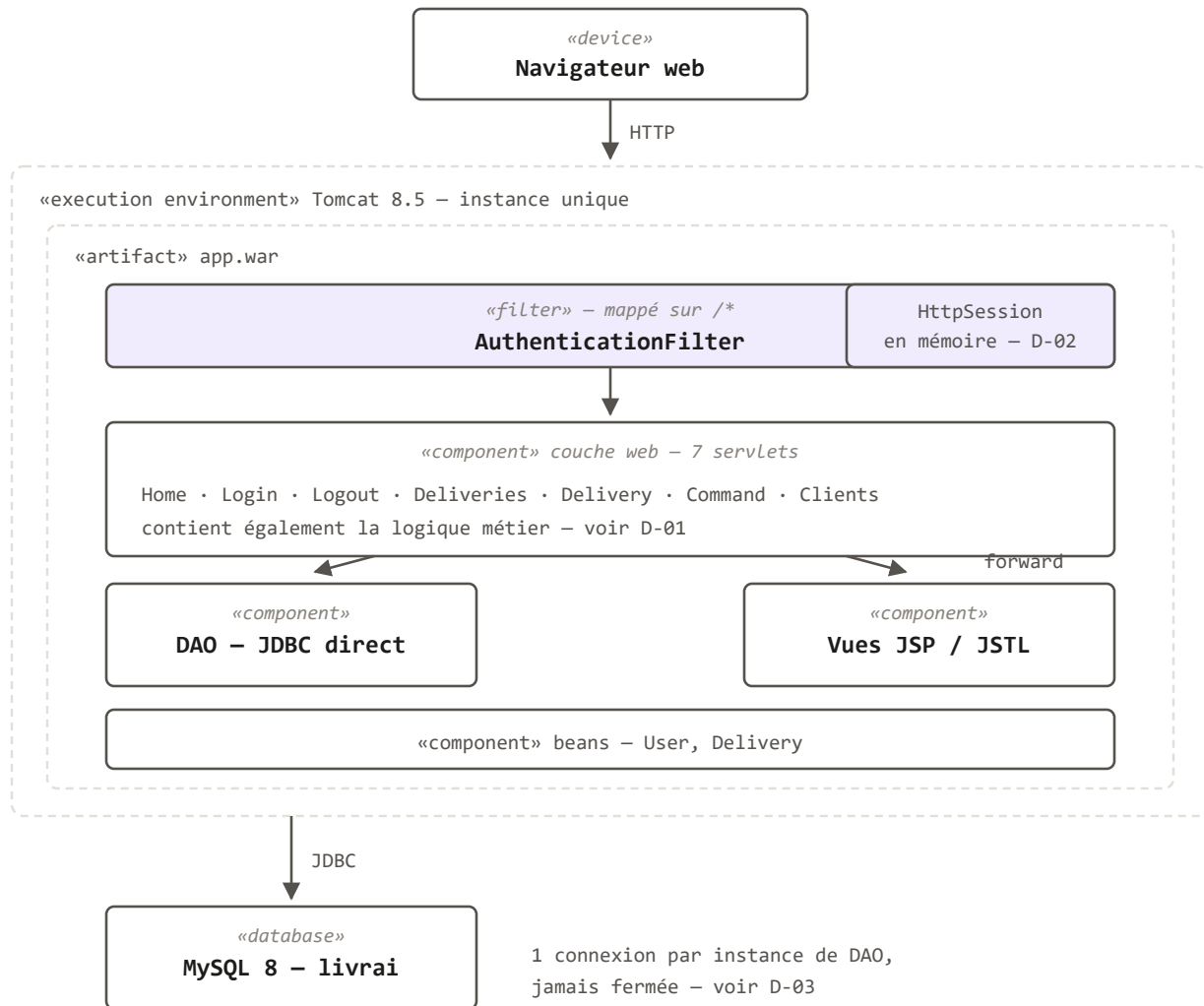
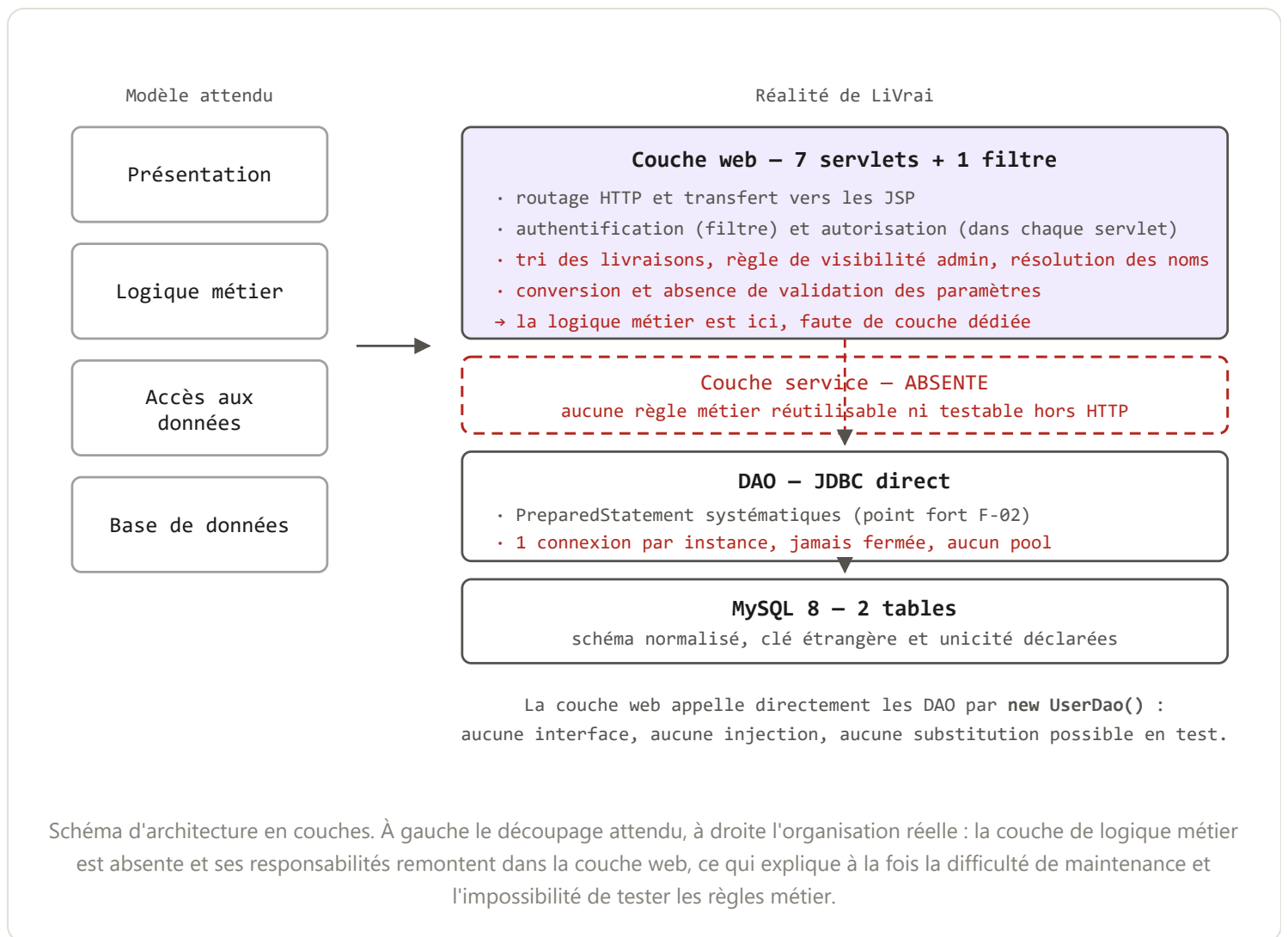


Diagramme de composants UML. L'application est un monolithe à trois couches physiques (web, données, base) mais sans couche service : la logique métier réside dans les servlets. Le stockage de session en mémoire et la gestion des connexions sont les deux points de fragilité structurels.

Lecture en couches

Rapportée au découpage classique en quatre couches — présentation, logique métier, accès aux données, base — l'architecture de LiVrai présente une anomalie centrale : **la couche de logique métier n'existe pas en tant que telle**. Ses responsabilités sont absorbées par la couche de présentation.



Mesures relevées sur l'application en fonctionnement

L'application a été déployée et sollicitée afin d'objectiver les constats. Les valeurs ci-dessous proviennent de cet environnement de test.

GRANDEUR	VALEUR RELEVÉE	PORTÉE
Connexions MySQL ouvertes après une session de test	60 , toutes <code>Sleep</code>	La plus ancienne ouverte depuis 897 s. Aucune fermeture par l'application.
Plafond de threads Tomcat	<code>maxThreads</code> = 150	Détermine le nombre maximal de connexions simultanées.
Plafond de connexions MySQL	<code>max_connections</code> = 151	Marge d'une seule connexion avant refus de service.
Délai de récupération des connexions	<code>wait_timeout</code> = 28 800 s	8 heures : sans effet utile pendant un pic d'activité.

GRANDEUR	VALEUR RELEVÉE	PORTÉE
Poids de <code>/livraisons</code> avec 20 004 livraisons	15,6 Mo	20 006 lignes de tableau transmises à chaque affichage. Environ 3 Ko avec 4 livraisons.
Mémoire Tomcat sous 10 requêtes concurrentes	242 Mo / 512 Mo	Tas à demi consommé par dix requêtes seulement sur ce volume.
Index sur <code>delivery.status</code>	Aucun	Le tri par statut s'effectue en mémoire Java, non en SQL.

Cheminement d'une requête

1. Le navigateur émet une requête HTTP vers une route déclarée dans `web.xml`.
2. `AuthenticationFilter`, mappé sur `/*`, intercepte. Il lit `userId` dans la session HTTP puis **interroge la base** pour recharger l'utilisateur — une requête SQL supplémentaire à chaque requête HTTP, y compris pour les ressources statiques.
3. Si l'utilisateur est authentifié ou la route publique (`/login` , `/logout`), la chaîne se poursuit ; sinon la requête est redirigée vers le formulaire de connexion.
4. Le servlet cible instancie directement ses DAO (`new UserDao()`), exécute la logique métier, place les résultats en attributs de requête.
5. Le servlet transfère vers une JSP qui produit la page HTML complète.

Organisation du code

PAQUET	CLASSES	RESPONSABILITÉ
<code>com.livrai</code>	1	Filtre d'authentification
<code>com.livrai.servlet</code>	7	Couche web et logique métier
<code>com.livrai.dao</code>	3	Accès aux données en JDBC
<code>com.livrai.bean</code>	2	Objets de transport, sans comportement

13	~700	0	0
-----------	-------------	----------	----------

Analyse critique

Points forts

Le CRM remplit sa fonction et soutient l'activité de LiVrai depuis sa mise en service. Ces qualités sont réelles et méritent d'être préservées dans toute évolution future.

F-01 Une base de code compacte et lisible

Treize classes, environ 700 lignes. L'absence de framework rend le cheminement d'une requête entièrement traçable à la lecture, sans magie ni configuration implicite. Un développeur peut prendre en main l'ensemble en une journée — un atout considérable pour une reprise ou une réécriture.

F-02 Requêtes SQL correctement paramétrées

Tous les accès à la base passent par des `PreparedStatement` avec paramètres liés. **L'application n'est pas vulnérable à l'injection SQL** — la faille la plus répandue de cette génération d'applications. C'est un point de rigueur notable.

F-03 Modèle de données propre

Schéma normalisé, clé étrangère déclarée entre `delivery` et `user`, contrainte d'unicité sur l'email, types appropriés (`DECIMAL` pour les montants, et non un flottant). Ce modèle est directement réutilisable dans une refonte.

F-04 Séparation des vues et échappement systématique

Les JSP sont placées sous `WEB-INF/`, donc inaccessibles directement par URL. Les données dynamiques sont rendues via `<c:out>`, qui échappe le HTML : l'application est protégée contre l'injection de scripts (XSS) dans les listes.

F-05 Sécurité par défaut sur le routage

Le filtre est mappé sur `/*` avec une liste blanche explicite de deux routes publiques. Toute nouvelle page est donc protégée par défaut — l'inverse d'un modèle où l'on doit penser à sécuriser chaque ajout.

F-06 Un périmètre fonctionnel maîtrisé

Le CRM fait peu de choses, mais ce qu'il fait correspond au métier. Cette clarté fonctionnelle facilite la spécification d'une cible : le besoin est connu et stable.

Déficiences

Les déficiences sont classées par criticité au regard des deux enjeux exprimés par LiVrai : la **volumétrie** et la **disponibilité**.

D-01 Absence de couche service : la logique métier vit dans les servlets

CRITIQUE

`DeliveriesServlet.doGet()` effectue la répartition entre livraisons à venir et passées, applique la règle de visibilité admin/client et résout les noms de clients — le tout dans la couche web. `DeliveryServlet` porte le contrôle d'autorisation en dur dans son `doPost`.

CONSÉQUENCE SUR LA MAINTENANCE

Aucune règle métier n'est réutilisable ni testable indépendamment de HTTP. Toute évolution oblige à modifier la couche web, avec un risque de régression non couvert par des tests.

CONSÉQUENCE SUR L'ÉVOLUTION

Exposer une API (application mobile, intégration client) imposerait de réécrire ces règles, ou de les dupliquer.

D-02 État de session en mémoire : la montée en charge horizontale est impossible

CRITIQUE – ENJEU DE DISPONIBILITÉ

L'identité de l'utilisateur connecté est stockée dans la `HttpSession`, c'est-à-dire dans la mémoire vive de l'instance Tomcat qui a traité la connexion. L'application est donc à **état** (*stateful*).

SCÉNARIO DE RUPTURE

Un vendredi de forte activité, LiVrai ajoute deux serveurs derrière un répartiteur de charge pour absorber le trafic. Une gestionnaire se connecte : sa session est créée sur le serveur 1. Sa requête suivante est distribuée au serveur 3, qui ne connaît pas cette session. **Elle est renvoyée vers l'écran de connexion en pleine saisie de commande.**

La conclusion est structurelle : la réplication du service est inopérante en l'état. La seule voie d'absorption d'un pic est d'augmenter la puissance de la machine unique (montée en charge *verticale*), qui se heurte à un plafond physique et coûteux.

CONSÉQUENCE SUR LA DISPONIBILITÉ

L'instance Tomcat unique constitue un **point de défaillance unique**. Son arrêt — panne, redémarrage, déploiement — rend le CRM totalement indisponible. Aucune redondance n'est possible sans traiter ce point au préalable.

EFFET SECONDAIRE

Tout redéploiement déconnecte l'ensemble des utilisateurs, les sessions en mémoire étant perdues.

`AbstractDao` conserve une `Connection` comme champ d'instance, ouverte à la demande et **jamais fermée**. Les servletsinstancient un DAO à chaque requête (`new UserDao()`), parfois plusieurs fois dans la même méthode. Aucun pool de connexions n'est configuré.

MESURÉ SUR L'APPLICATION

L'interrogation de `information_schema.PROCESSLIST` après une session de test révèle **60 connexions ouvertes à la base, toutes à l'état `Sleep`**, la plus ancienne datant de 897 secondes. Aucune n'est refermée par l'application : chaque thread Tomcat conserve la sienne aussi longtemps qu'il reste vivant.

LE CALCUL DE SATURATION

La configuration en place rend le seuil de rupture calculable : **Tomcat est configuré à `maxThreads = 150` et MySQL à `max_connections = 151`**. Une connexion étant retenue par thread actif, une charge saturant le pool de threads ouvre jusqu'à 150 connexions simultanées — **pour un plafond de 151. La marge est d'une seule connexion**. Le paramètre `wait_timeout` valant 28 800 secondes (8 heures), les connexions inutilisées ne sont pas récupérées à une échelle de temps utile pendant un pic.

CONSÉQUENCE À CHARGE ÉLEVÉE

Au-delà de ce seuil, MySQL refuse toute nouvelle connexion et **l'application devient entièrement indisponible** — sans que le processeur ou la mémoire de la machine soient nécessairement sollicités. La panne survient donc bien avant les limites matérielles du serveur, ce qui la rend contre-intuitive à diagnostiquer.

AGGRAVANT

Le filtre d'authentification exécutant un `SELECT` à chaque appel HTTP, ce coût est payé pour la moindre page, y compris celles ne nécessitant aucune donnée.

`getAllDeliveries()` exécute `SELECT ... FROM delivery` sans clause `LIMIT`. La totalité de l'historique est chargée en mémoire Java, puis parcourue, puis rendue en HTML — à chaque affichage de l'écran principal.

MESURÉ SUR L'APPLICATION

Un jeu de **20 004 livraisons** a été injecté en base, puis l'écran `/livraisons` a été appelé en tant qu'administrateur. La page produite pèse **15,6 Mo** et contient **20 006 lignes de tableau**, transmises intégralement au navigateur à chaque affichage. À titre de comparaison, la même page avec 4 livraisons pèse environ 3 Ko.

PROGRESSION DU COÛT

Le poids de la page croît linéairement avec le volume total de livraisons, sans palier ni plafond. Ce comportement est **indépendant du nombre d'utilisateurs** : il se dégrade avec le seul historique, que LiVrai accumule par construction.

INDEX : CONSTAT NUANCÉ

Un index existe sur `delivery.userId` — non par choix de conception, mais parce que MySQL en crée un automatiquement pour toute clé étrangère. En revanche, **aucun index ne couvre `delivery.status`**, alors que le statut est le critère de répartition entre livraisons à venir et passées. Ce tri est d'ailleurs effectué en mémoire Java plutôt qu'en SQL, ce qui rend l'index inopérant même s'il existait.

AMPLIFICATION MÉMOIRE

Chaque requête matérialise toutes les livraisons en objets Java avant d'en produire le HTML. Dix requêtes concurrentes sur le jeu de 20 000 lignes portent la mémoire du processus Tomcat à 242 Mo pour un tas plafonné à 512 Mo. À charge plus élevée, l'épuisement du tas devient un risque concret.

D-05 Mots de passe stockés et comparés en clair

CRITIQUE – SÉCURITÉ

`LoginServlet` authentifie par `user.getPassword().equals(password)`. Les mots de passe sont donc enregistrés en base sans hachage, et comparés tels quels. Le script d'initialisation le confirme : le compte administrateur est créé avec le mot de passe `'admin'` en clair.

VÉRIFIÉ SUR L'APPLICATION

Deux comptes clients ont été créés via l'interface d'administration, puis la table `user` a été relue directement : **les mots de passe y figurent en clair**, exactement tels qu'ils ont été saisis dans le formulaire.

CONSÉQUENCE

Toute lecture de la base — sauvegarde égarée, accès d'un prestataire, intrusion — expose l'intégralité des mots de passe en clair. Les clients réutilisant leurs identifiants ailleurs sont exposés au-delà du périmètre de LiVrai.

AGGRAVANT

La comparaison par `equals()` n'est pas à temps constant, et `getUserById()` exécute `SELECT *`, faisant circuler le mot de passe dans l'application à chaque requête HTTP.

PORTÉE

Ce point dépasse le cadre technique : il engage la responsabilité de LiVrai vis-à-vis du RGPD.

D-06 Identifiants de connexion codés en dur

CRITIQUE – SÉCURITÉ ET EXPLOITATION

`AbstractDao.loadDatabase()` contient l'URL, l'utilisateur `root` et le mot de passe de la base directement dans le code source, donc versionnés dans le dépôt Git.

CONSÉQUENCE SÉCURITÉ

Le secret est exposé à toute personne ayant accès au dépôt, et l'application se connecte avec le compte le plus privilégié de MySQL au lieu d'un compte restreint.

CONSÉQUENCE EXPLOITATION

Changer d'environnement (test, préproduction, production) exige de **recompiler l'application**. Cela interdit toute chaîne de déploiement automatisée et rend la rotation du mot de passe impraticable.

D-07 Aucun test automatisé, et un code non testable

CRITIQUE – MAINTENANCE

JUnit est déclaré dans le `pom.xml` mais aucun test n'existe. Plus fondamentalement, **le code ne pourrait pas être testé en l'état** : les servletsinstancient leurs DAO par `new`, les DAO n'implémentent aucune interface, et la connexion à la base est établie dans la classe elle-même. Aucune substitution n'est possible.

CONSÉQUENCE

Chaque modification comporte un risque de régression non détectable autrement que manuellement. C'est le facteur principal du coût de maintenance signalé par les équipes — et le premier obstacle à une refonte progressive et sûre.

D-08 **Traitement des erreurs absent ou trompeur** MAJEUR

Trois comportements distincts, tous problématiques :

- `catch (ClassNotFoundException e) {}` — exception **totale**ment **ignorée**. Si le pilote MySQL est absent, l'application continue son exécution et échoue plus loin, sur un symptôme sans rapport avec la cause.
- `e.printStackTrace()` en cas d'échec de connexion : la trace part sur la sortie standard, `connection` reste à `null`, et l'erreur réelle survient ailleurs.
- Toute `SQLException` est convertie en `RuntimeException` nue, sans contexte métier.

CONSÉQUENCE

Le diagnostic d'un incident en production est long et incertain. Aucun système de journalisation n'est en place — ni SLF4J, ni Log4j — ce qui empêche toute supervision automatisée.

D-09 **Autorisation dispersée et incomplète** MAJEUR – SÉCURITÉ

Le filtre gère l'**authentification** (« qui es-tu ? ») mais l'**autorisation** (« as-tu le droit ? ») est éparpillée dans chaque servlet, sous forme de tests `if (user.isAdmin())`.

FAILLE IDENTIFIÉE

`ClientsServlet` **ne vérifie pas le rôle**, contrairement à `DeliveryServlet` qui le fait. Tout utilisateur authentifié, y compris un client, peut appeler `/clients`.

EXPLOITATION VÉRIFIÉE

Test conduit sur l'application en fonctionnement avec un compte client (`admin = false`) : l'appel à `/clients` retourne **la liste complète des clients de LiVrai avec leurs adresses électroniques**, y compris celles d'entreprises concurrentes. Le même compte a pu **créer un nouvel utilisateur** en base par un `POST` sur cette route. Il s'agit d'une fuite de données commerciales doublée d'une élévation de privilège.

CAUSE STRUCTURELLE

Le contrôle reposant sur la vigilance du développeur à chaque ajout, l'oubli est le comportement par défaut. Un modèle d'autorisation centralisé supprimerait cette classe d'erreur.

D-10 Aucune validation des données entrantes MAJEUR

`CommandServlet` convertit directement les paramètres HTTP :

`Integer.valueOf(request.getParameter("volume"))` . Aucune vérification de présence, de format ni de plage.

VÉRIFIÉ SUR L'APPLICATION

Deux commandes ont été soumises depuis un compte client. Volume `-99` et poids `-50` : acceptés, `HTTP 200` , **enregistrés en base** et affichés dans la liste des livraisons à traiter. Volume `abc` : `HTTP 500` , page d'erreur brute du serveur exposée à l'utilisateur.

CONSÉQUENCE

Des états métier invalides entrent en base sans obstacle et remontent jusqu'à l'administrateur, qui peut les accepter et les facturer. Une valeur non numérique interrompt le parcours sur une erreur technique non traitée.

IDEM À LA FACTURATION

`DeliveryServlet` accepte n'importe quel prix, sans borne ni contrôle de cohérence.

D-11 Socle technique en fin de vie MAJEUR

Java 1.6 comme cible de compilation, `javax.servlet` 3.0 et Tomcat 8.5 forment un ensemble dont le support a pris fin. Aucun correctif de sécurité n'est plus publié pour le serveur d'application.

CONSÉQUENCE SUR LE RECRUTEMENT

Ces technologies ne sont plus enseignées ni pratiquées : trouver un développeur disposé à les maintenir devient difficile et coûteux.

CONSÉQUENCE SUR L'ÉCOSYSTÈME

La migration `javax` → `jakarta` ayant eu lieu dans tout l'écosystème Java, les bibliothèques récentes sont incompatibles avec cette base. L'application est enfermée dans son état actuel.

D-12 Absence de transactions MAJEUR

Chaque opération JDBC s'exécute en validation automatique. Aucune démarcation transactionnelle n'existe. Le périmètre fonctionnel actuel n'enchaîne jamais deux écritures, ce qui limite le risque immédiat — mais toute évolution qui le ferait (facturer et notifier, créer un client et sa première commande) exposerait la base à des états incohérents.

D-13 Défauts fonctionnels et de rendu en production MAJEUR

Le bouton « Refuser » poste vers une route qui ne traite pas les écritures : le refus d'une commande n'est jamais enregistré (UX-06). S'y ajoutent les décalages de colonnes du tableau des livraisons (UX-05). Ces anomalies sont indépendantes de l'architecture et corrigeables immédiatement, sans attendre la refonte.

D-14 Statuts métier stockés en texte libre français MINEUR

Le statut est un `VARCHAR(255)` contenant `"Acceptée"`, `"En attente"` ... Les comparaisons se font par égalité de chaînes, y compris dans les gabarits JSP (`#{delivery.status == 'Acceptée'}`).

CONSÉQUENCE

Aucune contrainte d'intégrité ne protège ce champ : une faute de frappe crée un état invalide et silencieux. La logique métier dépend de la langue d'affichage, ce qui rend toute internationalisation impossible sans migration de données. Les accents dans les comparaisons exposent en outre à des erreurs d'encodage.

L'essentiel en six points

Au-delà du relevé exhaustif, trois forces et trois risques déterminent l'avenir du système au regard de la croissance de LiVrai.

LES TROIS FORCES À PRÉSERVER

1. **Un modèle de données sain** (F-03). Schéma normalisé, intégrité référentielle et unicité déclarées, type `DECIMAL` pour les montants. Il est réutilisable en l'état dans toute cible et constitue un acquis, non un coût.
2. **Une base de code intégralement analysable** (F-01). Treize classes, environ 700 lignes, aucun framework masquant le comportement. Une reprise complète est possible en quelques jours, ce qui réduit considérablement le risque d'une refonte.
3. **Deux risques de sécurité majeurs déjà écartés** (F-02, F-04). Requêtes systématiquement paramétrées : aucune injection SQL. Échappement systématique dans les vues : aucune injection de script. Ces deux propriétés sont à conserver explicitement dans la cible.

LES TROIS RISQUES DÉTERMINANTS

1. **Le service ne peut pas être répliqué** (D-02). L'état d'authentification résidant en mémoire, ajouter des serveurs déconnecte les utilisateurs. LiVrai ne peut donc ni absorber un pic par la montée en charge horizontale, ni supprimer son point de défaillance unique. *Lien direct avec l'exigence de disponibilité.*
2. **La saturation survient à 150 connexions, pour un plafond de 151** (D-03). Le seuil est mesuré et la marge nulle. La panne se produit avant toute limite matérielle, ce qui la rend difficile à anticiper par la supervision usuelle. *Lien direct avec l'exigence de disponibilité.*
3. **La dégradation par l'historique est continue et sans palier** (D-04). 15,6 Mo par affichage pour 20 000 livraisons, sans pagination ni limite. Chaque commande enregistrée alourdit définitivement l'écran principal. *Lien direct avec l'exigence de volumétrie.*

Facteur aggravant transverse — l'absence de tests et de couche service (D-01, D-07) ne crée pas de panne par elle-même, mais rend toute correction de ces trois risques coûteuse et incertaine. C'est ce qui transforme un problème technique en problème de trajectoire.

Synthèse des risques

RÉF.	DÉFICIENCE	ENJEU LIVRAI	EFFET DE LA CROISSANCE
D-02	Session en mémoire	DISPONIBILITÉ	Bloque toute redondance. Le risque augmente avec la criticité du service.
D-03	Connexions non gérées	DISPONIBILITÉ	Saturation de MySQL à charge concurrente : panne totale.
D-04	Requêtes non bornées	VOLUMÉTRIE	Dégradation continue avec l'historique, sans palier.
D-05	Mots de passe en clair	SÉCURITÉ	Impact proportionnel au nombre de clients enregistrés.
D-06	Secrets dans le code	SÉCURITÉ	Empêche l'industrialisation du déploiement.
D-09	Fuite de données via /clients	SÉCURITÉ	Exposition croissante avec le portefeuille clients.
D-01 , D-07	Pas de couche service ni de tests	MAINTENANCE	Coût et risque de chaque évolution en hausse continue.

RÉF.	DÉFICIENCE	ENJEU LIVRAI	EFFET DE LA CROISSANCE
D-11	Socle en fin de vie	PÉRENNITÉ	Difficulté de recrutement et de mise à jour croissante.

06

Conclusion

Le CRM LiVrai est une application fonctionnelle, compacte et compréhensible, dont l'architecture appartient à une génération technique révolue. Elle ne peut, en l'état, ni absorber un pic de charge ni supporter la croissance de l'historique des livraisons.

Réponse aux trois signaux

SIGNAL	VÉRIFIÉ	MÉCANISME ÉTABLI
Lenteurs	CONFIRMÉ	Trois causes cumulées : connexions ouvertes sans pool à chaque requête (D-03), chargement intégral des tables sans LIMIT — 15,6 Mo mesurés pour 20 000 livraisons (D-04), et rechargement complet de la page à chaque interaction (UX-01).
Maintenance difficile	CONFIRMÉ	Logique métier enfermée dans la couche web (D-01), aucun test et code non testable (D-07), socle sans support (D-11). Chaque évolution se fait sans filet.
Disponibilité en pic	RISQUE AVÉRÉ	L'état de session en mémoire interdit la réplication (D-02) : l'instance unique est un point de défaillance unique. La gestion des connexions fixe par ailleurs un plafond mesuré — 150 connexions possibles pour 151 autorisées — au-delà duquel le service devient indisponible avant toute limite matérielle (D-03).

Ce qui est solide et doit être conservé

Le modèle de données est propre et directement réutilisable. Le périmètre fonctionnel est clair, stable et compris des équipes. Les requêtes paramétrées et l'échappement systématique dans les vues témoignent d'une rigueur réelle sur deux risques majeurs. La compacité du code rend l'ensemble intégralement analysable — ce qui est un avantage rare au moment d'engager une refonte.

Les trois natures de problèmes

Toutes les déficiences ne relèvent pas du même traitement, et cette distinction conditionne la suite :

1. **Ce qui se corrige immédiatement, sans refonte** — le bouton « Refuser » inopérant (D-13), l'absence de contrôle de rôle sur `/clients` (D-09), les décalages de colonnes. Ces défauts affectent la production aujourd'hui et ne dépendent d'aucun choix d'architecture.
2. **Ce qui relève d'un choix structurel** — le stockage de session (D-02), l'absence de couche service (D-01), la gestion des connexions (D-03), le stockage des mots de passe (D-05). Ces points ne se corrigent pas ponctuellement : ils découlent de la conception d'ensemble.
3. **Ce qui relève de l'obsolescence** — Java 1.6, `javax.servlet`, Tomcat 8.5 (D-11). Ces éléments imposent une décision de trajectoire, indépendamment des deux autres catégories.

Conclusion générale

L'application a rempli son office à l'échelle pour laquelle elle a été conçue. Les difficultés rencontrées ne traduisent pas un défaut de réalisation, mais l'écart entre cette échelle initiale et celle qu'atteint aujourd'hui LiVrai.

Deux constats commandent la suite. Le premier est que **la disponibilité en pic ne peut pas être obtenue par un ajustement** : tant que l'état d'authentification réside dans la mémoire du serveur, la redondance est inopérante et le service reste exposé à une interruption totale. Le second est que **la dégradation liée à la volumétrie est continue et sans palier** : chaque livraison supplémentaire alourdit l'écran principal, sans qu'aucun seuil ne vienne stabiliser la situation.

Ces deux constats, associés à l'absence de tests qui rend toute modification risquée, établissent que les enjeux exprimés par LiVrai relèvent de la conception d'ensemble et non d'optimisations ponctuelles. Le présent audit fournit la base factuelle nécessaire à l'arbitrage de la trajectoire technique, qui fait l'objet des travaux suivants.

Recommandation préalable : les défauts fonctionnels identifiés en production (D-09, D-13) affectent les utilisateurs et la confidentialité des données dès aujourd'hui. Leur correction ne dépend d'aucune décision de refonte et devrait être engagée sans attendre.

Méthode. Audit conduit en deux volets. D'une part l'analyse statique du code source, du schéma de base de données et des descripteurs de déploiement, chaque constat étant rattaché à un élément identifiable du code. D'autre part le **déploiement et l'exercice de l'application** — MySQL 8 en conteneur, Tomcat 8.5, archive WAR — afin de parcourir les deux profils utilisateurs, de vérifier les comportements et de relever les mesures citées (connexions à la base, poids de page, consommation mémoire, paramètres de configuration).

Réserve. Les mesures proviennent d'un environnement de test alimenté par des données générées, non de la production. Les valeurs de configuration relevées (`maxThreads`, `max_connections`) correspondent aux réglages par défaut et peuvent différer en production — le rapport de saturation qu'elles établissent reste néanmoins structurel. Un test de charge sur une copie représentative de la base permettrait de confirmer le point de rupture réel et de le situer par rapport au trafic effectif de LiVrai.